Using Idle GPU Time While Tools Run: Deadline Scheduling for Asynchronous LLM Agents

[Author names] [1]

Abstract

An LLM agent that launches several tool calls in parallel leaves its serving slot idle while the tools run, and then must read every result before a completion deadline. We build a deadline-aware agent runtime in front of vLLM that decides, as results arrive, what the GPU should do with that idle time: summarize results to shrink the final prompt, prefill raw results into the KV cache (*preloading*), or wait. Decisions come from an exact online MDP over a profiled token-cost model and from a simple distilled rule (RSS). On Qwen3-8B on an A40 with HotpotQA and randomized tool delays, in a context-constrained stress setting, the scheduled policies miss 0/10 tasks at a 16 s deadline versus 7–9/10 for fixed policies, because they summarize while tools are still running and leave 1.2 s of summarization after the last tool return instead of 4.2–5.7 s. With an unconstrained context the same policies correctly stop summarizing. KV preloading, verified with server-side cached-token counts, keeps final-answer latency at $\approx$0.5 s regardless of evidence size (vs. 2.3 s for 10.8k tokens cold); with 8 searches per question it finishes every task earlier (1.6 s on average, 20/20 questions) and cuts the miss rate at 16 s from 45% to 15%.

1. Problem

Agents increasingly issue tool calls asynchronously and in parallel (Kim et al., 2024): search, code execution, and API calls whose latencies are long and unpredictable compared with a model step. In an interactive or SLO-bound deployment, the whole task (planning, tool calls, reading results, and the final answer) must finish by a deadline. Two serving-side facts make this a scheduling problem. (1) While tools are running the model is idle. Any model work done then is free; the same work done after the last tool returns sits directly on the critical path. (2) Results cannot always be read raw. Long results exceed the context budget, or make the final prefill slow, so some must be *summarized* (a generation costing seconds of decode) or their prefill must be *precomputed*.

Question. For tool results that arrive at random times and feed one serialized model slot, when should the runtime summarize results (one at a time or in batches), prefill them into the KV cache, or leave them raw for the final answer, so the task finishes before its deadline?

Why it is new. Serving systems manage the KV cache of requests that pause for external calls (InferCept (Abhyankar et al., 2024)), schedule multi-call LLM applications (Parrot (Lin et al., 2024)), and reuse shared prefixes (SGLang (Zheng et al., 2024), vLLM automatic prefix caching (Kwon et al., 2023)). These optimize throughput or latency of the calls they are given. We instead choose *which model work to create* from the results of in-flight tools and *when to run it*, against a hard end-to-end deadline, using the idle time that tool waits create.

Challenges. Return times are unknown when decisions are made; the model slot is non-preemptive, so a 3 s summary started just before the last return delays the answer; the right choice depends on context limits and on measured prefill/decode costs; and the real system must match the model closely enough for model-based decisions to transfer.

2. Design

Runtime. One event loop per task owns all state. The agent plans n search queries (one model call), launches all n tools at once, and then repeatedly asks a pluggable *policy* for the next action: SUMMARIZE(B) (one structured-output call that emits three facts per result for a batch B of returned results), PRELOAD(B) (a prefix-only request with one output token that prefills raw results into vLLM's prefix cache), WAIT, or FINAL. One timeout covers the whole task, including planning, controller time, and the final answer. Runtime guards enforce one generation at a time, the context cap, and that every result is consumed exactly once. Every event is traced (JSONL), from which

[1]Harvard University, CS 243 (Fall 2026). Midterm report. Correspondence to: [Author] <[email]>.

Proceedings of the 42nd International Conference on Machine Learning, Vancouver, Canada. PMLR 267, 2025. Copyright 2025 by the author(s).

we compute a critical-path breakdown and serving metrics.

Cost model. We profiled 48 requests (prompts 243–2534 tokens, outputs 16–192) on the serving stack. Latency is affine in tokens: $\tau = \beta_p\, p + \beta_d\, o$ with $\beta_p = 0.18$ ms/token (prefill) and $\beta_d = 29.3$ ms/token (decode), $R^2 = 0.9999$, median relative error 0.4%. A summary of one result costs $\approx$3 s (102 output tokens) while it saves only $\beta_p(L - s) \approx 0.09$ s of final prefill per result: summarizing pays off only when the context forces it.

Theory. In a restricted model (one model slot, known count-based costs) we prove: an exact subset dynamic program for the offline problem; that if all results return together and fit, folding them raw is optimal; that fold is within $(n-1)\beta_p(L-s)$ of optimal whenever the raw prompt fits; and that some optimal schedule summarizes a release-order prefix in contiguous batches ($O(n^2)$ solver). For i.i.d. tool delays the state (time, #returned, #summarized) is sufficient, so the *optimal online policy* is a lookup table from backward induction over an MDP with Binomial return transitions (solved in under 0.1 s for $n = 4$). Inspecting it yields **RSS** (required-summary slack): summarize only the m results needed for the final prompt to fit, do it while other tools are still running, and fold the rest raw. With preloading, folding everything raw while prefilling each result as it arrives is optimal when the context fits.

KV preloading. The final prompt is an append-only evidence log (preloaded raw results in preload order, then summaries, then remaining raw results), so every preload request's prompt is a prefix of the final prompt and vLLM's automatic prefix caching reuses its KV blocks. Preload requests are charged to the deadline, preloaded results are never summarized, and the prefix cache is reset before every task so runs are independent. The `fold_preload` policy runs fold and turns every idle wait into a preload of newly returned results.

3. Initial evaluation

Setup. Qwen3-8B (Qwen Team, 2025) (bf16, thinking off), vLLM 0.32.0, one NVIDIA A40, one request at a time. Tasks are HotpotQA distractor questions (Yang et al., 2018): $n = 4$ BM25 searches over the question's 10 paragraphs (top-4 passages). Tool delays are injected per call and reproducible: Uniform[0.5,4] s, or with probability 0.3 a slow Uniform[4,12] s. Each run scores deadline-qualified F1 (F1 if on time, else 0). Deadlines (13/16/19 s) and the controller's costs were fixed on 8 separate calibration questions; results are on 10 held-out questions with 95% bootstrap intervals over questions.

Restricted-context stress setting. To make the context bind, results are capped at 576 tokens and the final prompt at 1600 tokens, so a typical question needs two summaries.

Table 1. Serving view at $D = 16$ s (restricted context; mean over the same 10 questions per policy; missed runs cut at the deadline). GPU util. = fraction of the tool-wait window in which the model is busy.

policy	missed	end (s)	GPU util. during tools	model s after last return	GPU s total
fold	7/10	14.6	0.00	4.36	6.5
incremental	7/10	15.7	0.67	5.48	13.3
batch of 2	9/10	15.9	0.43	5.70	11.6
RSS	0/10	11.9	0.58	1.71	8.6
optimal (MDP)	0/10	12.0	0.57	1.73	8.6

This is a deliberately constrained setting and we label it as such; the normal-context control follows.

Result. Figure 1 shows the main result. At $D = 16$ s the optimal policy and RSS miss 0/10 tasks; static fold misses 7/10, summarize-each-result (incremental) 7/10, and batches of two 9/10. The model's predicted miss rates track the measurements; the largest errors are for static policies near their transition (fold at 16 s: 0.53 predicted vs. 0.70 measured; batch-of-2 at 19 s: 0.23 vs. 0.40). The model uses a fixed planning time (median 1.65 s, max 2.7 s), which is one likely source; we have not yet isolated it.

Where the time goes. Table 1 breaks down the same 10 runs per policy at $D = 16$ s. Tool waiting is identical across policies (8.07 s; same injected delays). What differs is how much model work lands after the last return: fold leaves the GPU idle for the entire tool wait and then summarizes on the critical path, while RSS uses 58% of the tool wait for summaries and leaves 1.7 s of model time after the last return. Incremental also works during the wait, but because it summarizes *every* result, including the last ones to arrive, 5.5 s still remains after the last return.

Normal-context control. Removing the caps (results up to 1200 tokens, final prompt up to 7000) on the same questions and deadlines, RSS makes zero summaries in every run and matches fold exactly (miss 0.4/0/0 at 13/16/19 s), as the theory predicts, while incremental still pays for summaries (miss 1.0/0.7/0; mean completion 16.8 s vs. 10.9 s at 19 s). The scheduler therefore only adds work when the context forces it.

KV preloading. Figure 2 measures the final step with and without preloading for evidence logs of 1–16 results ($\approx$600 tokens each). Without preloading the final request's latency grows with evidence size (0.79 s at 2.8k tokens, 2.29 s at 10.8k); with preloading it stays at 0.39–0.51 s, and for 4–16 results 72–94% of the final prompt is served from cache. The cost is moved, not removed: for 4–16 results the preload requests take 0.44–3.06 s in total, which is free only if tool waits are long enough to hide it. In the live agent (4 calibration questions), each preload reused the KV cache of the previous log (e.g. 1830 prompt tokens, 1248

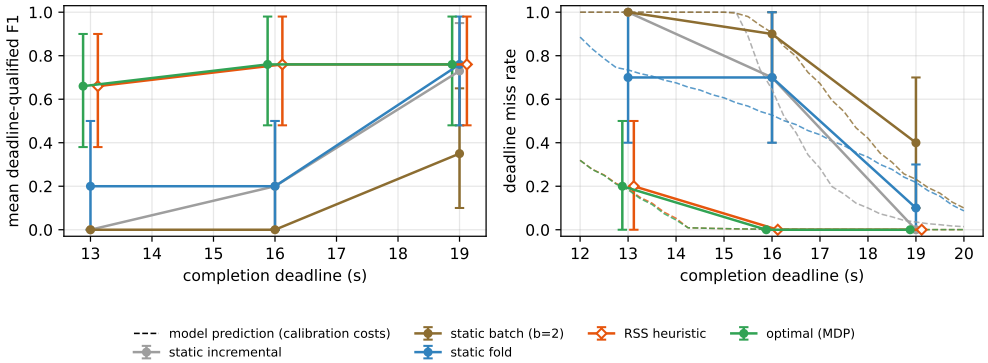


Figure 1. **Main result (restricted-context stress setting).** Deadline-qualified F1 (left) and miss rate (right) vs. deadline, 10 held-out HotpotQA questions, 150 runs on vLLM/A40. Dashed lines: miss rate predicted by the MDP from the calibration costs. RSS and the optimal MDP policy coincide in 29/30 question–deadline cells (points offset ± 0.12 s for visibility). F1 of on-time runs is similar for fold, RSS and optimal (0.77–0.78) and incremental (0.72); batch-of-2 scores 0.50 on its 7 on-time runs. The gain of RSS is timeliness, not answer quality.

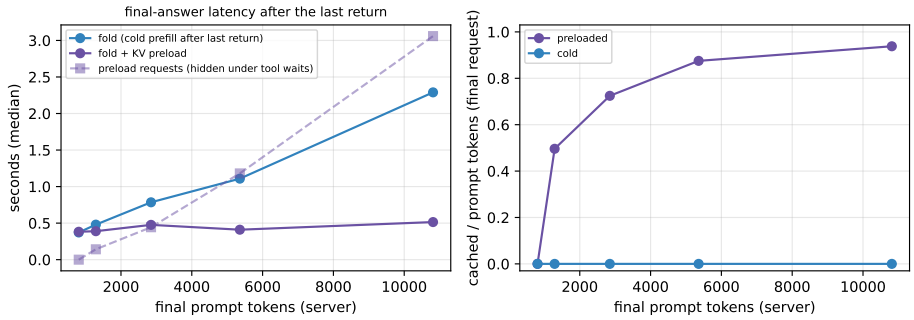


Figure 2. **KV preloading microbenchmark** (live server, prefix caching on, 30 requests). Left: latency of the final request after the last result arrives; preload requests (dashed) run while tools are still out. Right: fraction of the final prompt served from the prefix cache (server-reported `cached_tokens`).

cached; only the uncached tail is prefilled) and answers were unchanged; at the pilot's small evidence size the time after the last return dropped only from 0.60–0.72 s to 0.38–0.55 s.

Preloading under deadlines. We then scale the evidence: 8 parallel searches per question over a pooled corpus (the question's paragraphs plus 90 distractor paragraphs), top-10 passages per search, on 20 held-out questions with the context not binding. Fold, `fold_preload` and incremental never read the clock, so a single run per question with a non-binding deadline gives each completion time T, and the miss rate at *any* deadline is exactly $P(T > D)$ on the same runs. Preloading finished earlier on 20/20 questions, by 1.61 s on average (95% CI 1.49–1.72 s; range 1.10–2.06 s), with identical answers on all 20 (Figure 3). Model time after the last tool return fell from 2.35 s to 0.70 s (median), and at $D = 16$ s the miss rate fell from 0.45 to 0.15. The preload requests used 2.15 s of otherwise idle GPU time during tool waits (GPU utilization during tools 0% $\rightarrow$ 28%), more than the 1.6 s they saved, because each request pays a fixed overhead; this is acceptable only because that time was idle. Summarizing every result (incremental) is not competitive at this scale (30.3 s mean completion: 8 summaries of $\sim$3 s each).

4. Remaining challenges and plan

Limitations of the current evaluation. (1) The headline gain appears in a context-constrained setting we constructed; with an unconstrained context, summarization is unnecessary and the policies reduce to fold. The gain is real when context truly binds (many tools, long outputs, small context windows, multi-round agents), and showing that this regime is common is the main open question. (2) Ten held-out questions give wide intervals. (3) One request at a time; prefill is cheap for an 8B model on an A40, so preloading matters only with large evidence (0.15–0.3 s saved at the pilot's $\sim$2–3k-token prompts, 1.6 s at $\sim$11k). (4) In the normal-context run, summaries gave higher F1 than raw evidence (0.83 vs. 0.63 at 19 s), but this is 2/10 questions and needs a larger sample.

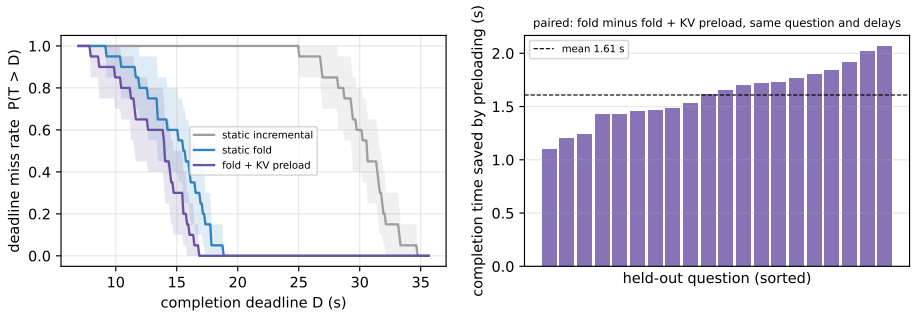


Figure 3. **KV preloading under deadlines** (20 held-out questions, 8 searches, ∼1.4k tokens per result, ∼11k-token final prompt; context not binding). Left: miss rate $P(T > D)$ for every deadline D, 95% bootstrap bands over questions. Right: paired completion-time saving per question.

Plan to the final report. (i) Scale to 30 held-out questions and two delay seeds, plus a regime with predictable delays. (ii) Combine RSS with preloading (the theory predicts that preloading can change which results to summarize) and evaluate it at larger evidence loads. (iii) Admission control: how many tools to launch given the deadline, using an exact small-n MDP. (iv) A second serving optimization: speculative decoding for the summary and answer generations. (v) A multi-round agent loop (think, call, read, repeat). (vi) Several agents sharing one GPU, where preloads and summaries compete with other requests; a vLLM change only if a measured bottleneck requires it.

References

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. INFERCEPT: Efficient intercept support for augmented large language model inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Kim, S., Moon, S., Tabrizi, R., Lee, N., Mahoney, M. W., Keutzer, K., and Gholami, A. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Lin, C., Han, Z., Zhang, C., Yang, Y., Yang, F., Chen, C., and Qiu, L. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.